LAB ASSIGNMENT 12.3

Name : A . Sai Ram

Hall Ticket No : 2303A52282

BATCH No : 42

TASK 01: Sorting Student Records for Placement Drive Scenario

PROMPT:

Generate students records of 20members (name, roll number, cgpa) and print them in sorted manner using quick sort and merge sort, also write a function which displays the top 10 students.

Compare the time complexity of both sorting algorithms in large datasets in form of docstrings.

CODE:

```
from heapq import merge
import random
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa
def generate_students(num_students):
    students = []
    for i in range(num_students):
        name = f"Student{i+1}"
        roll_number = i + 1
        cgpa = round(random.uniform(2.0, 4.0), 2)
        students.append(Student(name, roll_number, cgpa))
    return students
def quick_sort(students):
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [s for s in students if s.cgpa > pivot]
    middle = [s for s in students if s.cgpa == pivot]
    right = [s for s in students if s.cgpa < pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

```python
def merge_sort(students):
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left_half = merge_sort(students[:mid])
    right_half = merge_sort(students[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def display_top_students(students, top_n=10):
    top_students = students[:top_n]
    print("Top Students:")
    for student in top_students:
        print(f"Name: {student.name}, Roll Number: {student.roll_number}, CGPA: {student.cgpa}")
```

```python
# Generate student records
students = generate_students(20)
# Sort students using quick sort
sorted_students_quick = quick_sort(students)
print("Students sorted by CGPA using Quick Sort:")
for student in sorted_students_quick:
    print(f"Name: {student.name}, Roll Number: {student.roll_number}, CGPA: {student.cgpa}")
# Sort students using merge sort
sorted_students_merge = merge_sort(students)
print("\nStudents sorted by CGPA using Merge Sort:")
for student in sorted_students_merge:
    print(f"Name: {student.name}, Roll Number: {student.roll_number}, CGPA: {student.cgpa}")
display_top_students(sorted_students_merge)
```

```python
"""
Time Complexity:
- Quick Sort: O(n log n) on average, O(n^2) in the worst case (when the smallest or largest element is always chosen as the pivot).
- Merge Sort: O(n log n) in all cases (best, average, and worst) because it always divides the array into two halves and processes them independently.
In practice, Quick Sort is often faster than Merge Sort for smaller datasets due to lower constant factors and better cache performance.
However, Merge Sort is more stable and performs better on larger datasets or when the data is
stored on external storage (like disk) because it accesses data sequentially.
"""
```

OUTPUT:

```
Students sorted by CGPA using Quick Sort:
Name: Student2, Roll Number: 2, CGPA: 3.98
Name: Student3, Roll Number: 3, CGPA: 3.97
Name: Student6, Roll Number: 6, CGPA: 3.96
Name: Student8, Roll Number: 8, CGPA: 3.9
Name: Student19, Roll Number: 19, CGPA: 3.7
Name: Student4, Roll Number: 4, CGPA: 3.69
Name: Student17, Roll Number: 17, CGPA: 3.62
Name: Student15, Roll Number: 15, CGPA: 3.5
Name: Student13, Roll Number: 13, CGPA: 3.38
Name: Student18, Roll Number: 18, CGPA: 3.17
Name: Student1, Roll Number: 1, CGPA: 2.94
Name: Student10, Roll Number: 10, CGPA: 2.86
Name: Student20, Roll Number: 20, CGPA: 2.77
Name: Student5, Roll Number: 5, CGPA: 2.73
Name: Student9, Roll Number: 9, CGPA: 2.64
Name: Student16, Roll Number: 16, CGPA: 2.45
Name: Student12, Roll Number: 12, CGPA: 2.44
Name: Student11, Roll Number: 11, CGPA: 2.35
Name: Student14, Roll Number: 14, CGPA: 2.23
Name: Student7, Roll Number: 7, CGPA: 2.14
```

```
Students sorted by CGPA using Merge Sort:
Name: Student2, Roll Number: 2, CGPA: 3.98
Name: Student3, Roll Number: 3, CGPA: 3.97
Name: Student6, Roll Number: 6, CGPA: 3.96
Name: Student8, Roll Number: 8, CGPA: 3.9
Name: Student19, Roll Number: 19, CGPA: 3.7
Name: Student4, Roll Number: 4, CGPA: 3.69
Name: Student17, Roll Number: 17, CGPA: 3.62
Name: Student15, Roll Number: 15, CGPA: 3.5
Name: Student13, Roll Number: 13, CGPA: 3.38
Name: Student18, Roll Number: 18, CGPA: 3.17
Name: Student1, Roll Number: 1, CGPA: 2.94
Name: Student10, Roll Number: 10, CGPA: 2.86
Name: Student20, Roll Number: 20, CGPA: 2.77
Name: Student5, Roll Number: 5, CGPA: 2.73
Name: Student9, Roll Number: 9, CGPA: 2.64
Name: Student16, Roll Number: 16, CGPA: 2.45
Name: Student12, Roll Number: 12, CGPA: 2.44
Name: Student11, Roll Number: 11, CGPA: 2.35
Name: Student14, Roll Number: 14, CGPA: 2.23
Name: Student7, Roll Number: 7, CGPA: 2.14
Top Students:
Name: Student2, Roll Number: 2, CGPA: 3.98
Name: Student3, Roll Number: 3, CGPA: 3.97
Name: Student6, Roll Number: 6, CGPA: 3.96
Name: Student8, Roll Number: 8, CGPA: 3.9
Name: Student19, Roll Number: 19, CGPA: 3.7
Name: Student4, Roll Number: 4, CGPA: 3.69
Name: Student17, Roll Number: 17, CGPA: 3.62
Name: Student15, Roll Number: 15, CGPA: 3.5
Name: Student13, Roll Number: 13, CGPA: 3.38
Name: Student18, Roll Number: 18, CGPA: 3.17
```

JUSTIFICATION:

In this task, we have sorted the students list with both quick sort and merge sort, even though its both sorting process, way of doing it is different. Time complexity in both quick sort and merge sort is same which is O(n log n) for both best and average cases, whereas in worst case, time complexity in quick sort is O(n^2) meanwhile merge sort is O(n log n), so it is better to use merge sort.

TASK 02: Implementing Bubble Sort with AI Comments

PROMPT:

Implement bubble sort on a unsorted list of 10 random integers and print the sorted list, also provide required comments and also provide time complexity analysis in the end.

CODE:

```python
import random
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the list
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
# Generate a list of 10 random integers
unsorted_list = [random.randint(1, 100) for _ in range(10)]
print("Unsorted List:", unsorted_list)
# Sort the list using bubble sort
sorted_list = bubble_sort(unsorted_list)
print("Sorted List:", sorted_list)
"""
Time Complexity:
- The time complexity of Bubble Sort is O(n^2) in the worst and average cases, where n is the number of elements in the list.
 This occurs when the list is in reverse order.
- In the best case, when the list is already sorted, the time complexity is O(n)
because it only needs to make one pass through the list to check if it is sorted.
- Overall, Bubble Sort is not the most efficient sorting algorithm for large datasets,
and it is primarily used for educational purposes to illustrate the concept of sorting.
"""
```

OUTPUT:

```
PS C:\Users\VIGNANAND RAO\OneDrive\Desktop\AI_coding> &
Unsorted List: [70, 2, 40, 45, 44, 3, 10, 44, 99, 50]
Sorted List: [2, 3, 10, 40, 44, 44, 45, 50, 70, 99]
PS C:\Users\VIGNANAND RAO\OneDrive\Desktop\AI_coding>
```

JUSTIFICATION:

In the above task, we have successfully implemented bubble sort technique on an unsorted list of numbers, also provided the required comments which are essential for better understanding of the logic and algorithm. Time complexity of bubble sort in Best case is O(n) whereas in average case and worst case it is O(n^2) which takes more time as it must swap 2 numbers.

TASK 03: Quick Sort and Merge Sort Comparison

PROMPT:

Generate a code for sorting a list using quick sort and merge sort using recursion and print the sorted list, also provide comparison of both the algorithms on random, sorted and reverse sorted lists in form of docstrings.

CODE:

```
import random

from pandas import merge
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

```python
# Generate random, sorted, and reverse sorted lists
random_list = [random.randint(1, 100) for _ in range(10)]
sorted_list = sorted(random_list)
reverse_sorted_list = sorted(random_list, reverse=True)
print("Random List:", random_list)
print("Sorted List:", sorted_list)
print("Reverse Sorted List:", reverse_sorted_list)
# Sort the lists using quick sort and merge sort
sorted_random_quick = quick_sort(random_list)
sorted_random_merge = merge_sort(random_list)
sorted_sorted_quick = quick_sort(sorted_list)
sorted_sorted_merge = merge_sort(sorted_list)
sorted_reverse_quick = quick_sort(reverse_sorted_list)
sorted_reverse_merge = merge_sort(reverse_sorted_list)
print("\nSorted Random List (Quick Sort):", sorted_random_quick)
print("Sorted Random List (Merge Sort):", sorted_random_merge)
print("\nSorted Sorted List (Quick Sort):", sorted_sorted_quick)
print("Sorted Sorted List (Merge Sort):", sorted_sorted_merge)
print("\nSorted Reverse Sorted List (Quick Sort):", sorted_reverse_quick)
print("Sorted Reverse Sorted List (Merge Sort):", sorted_reverse_merge)
"""
Time Complexity Comparison:
- Quick Sort:
  - Average and Worst Case: O(n log n) when the pivot selection is good.
  - Worst Case: O(n^2) when the smallest or largest element is always chosen as the pivot (e.g., sorted or reverse sorted lists).
  - Best Case: O(n log n) when the pivot selection is optimal.
- Merge Sort:
  - Time Complexity: O(n log n) in all cases (best, average, and worst) because it always divides the array into two halves and processes them independently.
In practice, Quick Sort is often faster than Merge Sort for smaller datasets due to lower constant factors and better cache performance. However,
Merge Sort is more stable and performs better on larger datasets or when the data is stored on external storage (like disk) because it accesses data sequentially.
"""
```

OUTPUT:

```
PS C:\Users\VIGNANAND RAO\OneDrive\Desktop\AI_coding> & C:/Python314/python.exe  C:/
Random List: [39, 81, 20, 87, 97, 84, 89, 53, 80, 94]
Sorted List: [20, 39, 53, 80, 81, 84, 87, 89, 94, 97]
Reverse Sorted List: [97, 94, 89, 87, 84, 81, 80, 53, 39, 20]

Sorted Random List (Quick Sort): [20, 39, 53, 80, 81, 84, 87, 89, 94, 97]
Sorted Random List (Merge Sort): [20, 39, 53, 80, 81, 84, 87, 89, 94, 97]

Sorted Sorted List (Quick Sort): [20, 39, 53, 80, 81, 84, 87, 89, 94, 97]
Sorted Sorted List (Merge Sort): [20, 39, 53, 80, 81, 84, 87, 89, 94, 97]

Sorted Reverse Sorted List (Quick Sort): [20, 39, 53, 80, 81, 84, 87, 89, 94, 97]
Sorted Reverse Sorted List (Merge Sort): [20, 39, 53, 80, 81, 84, 87, 89, 94, 97]
PS C:\Users\VIGNANAND RAO\OneDrive\Desktop\AI_coding>
```

```
"""
Time Complexity:
- Quick Sort: O(n log n) on average, O(n^2) in the worst case (when the smallest or largest element is always chosen as the pivot).
- Merge Sort: O(n log n) in all cases (best, average, and worst) because it always divides the array into two halves and processes them independently.
In practice, Quick Sort is often faster than Merge Sort for smaller datasets due to lower constant factors and better cache performance.
However, Merge Sort is more stable and performs better on larger datasets or when the data is
stored on external storage (like disk) because it accesses data sequentially.
"""'''
```

JUSTIFICATION:

In this task, we have implemented both quick sort and merge sort using recursion on all 3 cases, random list which is average case, sorted list which is best case and reverse sorted list which is worst case.

Time complexity of quick sort is O(n log n) in both best case and average case whereas in worst case, it is O(n^2) but for merge sort, time complexity is O(n log n) for all the 3 best, average and worst cases.

TASK 04: (Real-Time Application – Inventory Management System)

PROMPT:

There is a stock list with attributes (product id, name, price, stock quantity), generate a list of 20 items and 2 functions, one for searching the product with productid or name and other for sorting the items by price or quantity for stock analysis. Suggest the best efficient searching and sorting algorithms for this use case, implement them, justify the choice based on dataset size, update frequency and performance requirements, in form of a table mapping (operation -> recommended algorithm -> justification).

CODE:

```
import random
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity
def generate_products(num_products):
    products = []
    for i in range(num_products):
        product_id = f"P{i+1:03d}"
        name = f"Product{i+1}"
        price = round(random.uniform(10.0, 100.0), 2)
        stock_quantity = random.randint(1, 100)
        products.append(Product(product_id, name, price, stock_quantity))
    return products
def search_product(products, query):
    for product in products:
        if product.product_id == query or product.name == query:
            return product
    return None
def sort_products(products, key):
    if key == "price":
        return sorted(products, key=lambda x: x.price)
    elif key == "stock_quantity":
        return sorted(products, key=lambda x: x.stock_quantity)
    else:
        raise ValueError("Invalid sorting key. Use 'price' or 'stock_quantity'.")
```

```python
# Generate a list of 20 products
products = generate_products(20)
# Search for a product by product ID or name
search_query = "P005"  # Example search query
found_product = search_product(products, search_query)
if found_product:
    print(f"Product found: ID={found_product.product_id}, Name={found_product.name}, Price={found_product.price}, Stock Quantity={found_product.stock_quantity}")
else:
    print("Product not found.")
# Sort products by price
sorted_by_price = sort_products(products, "price")
print("\nProducts sorted by price:")
for product in sorted_by_price:
    print(f"ID={product.product_id}, Name={product.name}, Price={product.price}, Stock Quantity={product.stock_quantity}")
# Sort products by stock quantity
sorted_by_stock = sort_products(products, "stock_quantity")
print("\nProducts sorted by stock quantity:")
for product in sorted_by_stock:
    print(f"ID={product.product_id}, Name={product.name}, Price={product.price}, Stock Quantity={product.stock_quantity}")
"""
| Operation                      | Recommended Algorithm              | Justification                                                                                |
|--------------------------------|------------------------------------|----------------------------------------------------------------------------------------------|
| Search by Product ID or Name   | Linear Search                      | The dataset size is small (20 items), and the search operation is infrequent,                |
|                                |                                    |  making linear search efficient and straightforward.                                         |
|--------------------------------|------------------------------------|----------------------------------------------------------------------------------------------|
| Sort by Price or Stock Quantity| Timsort (Python's built-in sorted) | Timsort is a hybrid sorting algorithm derived from merge sort and insertion sort,            |
|                                |                                    |optimized for real-world data. It performs well on small datasets and is stable,              |
|                                |                                    | making it suitable for sorting products by price or stock quantity.                          |
|--------------------------------|------------------------------------|----------------------------------------------------------------------------------------------|
"""
```

OUTPUT:

```
PS C:\Users\VIGNANAND RAO\OneDrive\Desktop\AI_coding> & C:/Python314/py
Product found: ID=P005, Name=Product5, Price=20.08, Stock Quantity=25

Products sorted by price:
ID=P013, Name=Product13, Price=10.07, Stock Quantity=46
ID=P017, Name=Product17, Price=14.28, Stock Quantity=79
ID=P005, Name=Product5, Price=20.08, Stock Quantity=25
ID=P007, Name=Product7, Price=20.62, Stock Quantity=42
ID=P016, Name=Product16, Price=22.1, Stock Quantity=48
ID=P001, Name=Product1, Price=35.29, Stock Quantity=85
ID=P008, Name=Product8, Price=37.33, Stock Quantity=32
ID=P002, Name=Product2, Price=41.83, Stock Quantity=19
ID=P015, Name=Product15, Price=48.14, Stock Quantity=77
ID=P019, Name=Product19, Price=50.4, Stock Quantity=8
ID=P009, Name=Product9, Price=50.77, Stock Quantity=61
ID=P004, Name=Product4, Price=53.55, Stock Quantity=88
ID=P018, Name=Product18, Price=57.82, Stock Quantity=52
ID=P006, Name=Product6, Price=58.96, Stock Quantity=44
ID=P003, Name=Product3, Price=62.64, Stock Quantity=11
ID=P010, Name=Product10, Price=71.0, Stock Quantity=45
ID=P014, Name=Product14, Price=71.05, Stock Quantity=7
ID=P012, Name=Product12, Price=77.71, Stock Quantity=61
ID=P020, Name=Product20, Price=80.72, Stock Quantity=49
ID=P011, Name=Product11, Price=86.01, Stock Quantity=4
```

```
Products sorted by stock quantity:
ID=P011, Name=Product11, Price=86.01, Stock Quantity=4
ID=P014, Name=Product14, Price=71.05, Stock Quantity=7
ID=P019, Name=Product19, Price=50.4, Stock Quantity=8
ID=P003, Name=Product3, Price=62.64, Stock Quantity=11
ID=P002, Name=Product2, Price=41.83, Stock Quantity=19
ID=P005, Name=Product5, Price=20.08, Stock Quantity=25
ID=P008, Name=Product8, Price=37.33, Stock Quantity=32
ID=P007, Name=Product7, Price=20.62, Stock Quantity=42
ID=P006, Name=Product6, Price=58.96, Stock Quantity=44
ID=P010, Name=Product10, Price=71.0, Stock Quantity=45
ID=P013, Name=Product13, Price=10.07, Stock Quantity=46
ID=P016, Name=Product16, Price=22.1, Stock Quantity=48
ID=P020, Name=Product20, Price=80.72, Stock Quantity=49
ID=P018, Name=Product18, Price=57.82, Stock Quantity=52
ID=P009, Name=Product9, Price=50.77, Stock Quantity=61
ID=P012, Name=Product12, Price=77.71, Stock Quantity=61
ID=P015, Name=Product15, Price=48.14, Stock Quantity=77
ID=P017, Name=Product17, Price=14.28, Stock Quantity=79
ID=P001, Name=Product1, Price=35.29, Stock Quantity=85
ID=P004, Name=Product4, Price=53.55, Stock Quantity=88
PS C:\Users\VIGNANAND RAO\OneDrive\Desktop\AI_coding>
```

Operation	Recommended Algorithm	Justification
Search by Product ID or Name	Linear Search	The dataset size is small (20 items), and the search operation is infrequent, making linear search efficient and straightforward.
Sort by Price or Stock Quantity	Timsort (Python's built-in sorted)	Timsort is a hybrid sorting algorithm derived from merge sort and insertion sort, optimized for real-world data. It performs well on small datasets and is stable, making it suitable for sorting products by price or stock quantity.

JUSTIFICATION:

In the above task we have using linear search for searching of the required product and we used Timsort(Python's built-in sorted function) for sorting the items based on product price and also based on product quantity (availability) which is essential for stock analysis. We have implemented both searching and sorting algorithms in the real world scenario.
Timsort is a build-in function of Python which hybrid sorting algorithm derived from merge sort and insertion sort, which is very efficient for real world use cases.

TASK 05: Real-Time Stock Data Sorting & Searching

PROMPT:

generate a code which quickly sort stock price data (Stock symbol, opening price, closing price) by daily gain/loss and search for specific stock symbol efficiently. Implement sorting algorithm to rank stocks by percentage change and implement a search function that retrieves stock data instantly. For sorting use Heap Sort only and for searching use Hash Maps only. Compare the performance with standard library functions (sorted(), dictlookups) and analyze trade-offs in form of docstrings.

CODE:

```
import random
class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol
        self.opening_price = opening_price
        self.closing_price = closing_price
    def percentage_change(self):
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[left].percentage_change() > arr[largest].percentage_change():
        largest = left
    if right < n and arr[right].percentage_change() > arr[largest].percentage_change():
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
    return arr
def search_stock(stocks, symbol):
    stock_map = {stock.symbol: stock for stock in stocks}
    return stock_map.get(symbol, None)
```

```python
# Generate random stock data
stocks = []
for i in range(20):
    symbol = f"STK{i+1:03d}"
    opening_price = round(random.uniform(100.0, 500.0), 2)
    closing_price = round(random.uniform(100.0, 500.0), 2)
    stocks.append(Stock(symbol, opening_price, closing_price))
# Sort stocks by percentage change using heap sort
sorted_stocks_heap = heap_sort(stocks)
print("Stocks sorted by percentage change (Heap Sort):")
for stock in sorted_stocks_heap:
    print(f"Symbol: {stock.symbol}, Opening Price: {stock.opening_price}, Closing Price: {stock.closing_price}, Percentage Change: {stock.percentage_change():.2f}%")
# Search for a specific stock symbol
search_symbol = "STK005"  # Example search symbol
found_stock = search_stock(stocks, search_symbol)
if found_stock:
    print(f"\nStock found: Symbol={found_stock.symbol}, Opening Price={found_stock.opening_price}, Closing Price={found_stock.closing_price}, Percentage Change={found_stock.percenta
else:
    print("\nStock not found.")
"""
```

OUTPUT:

```
Stocks sorted by percentage change (Heap Sort):
Symbol: STK019, Opening Price: 247.99, Closing Price: 101.76, Percentage Change: -58.97%
Symbol: STK017, Opening Price: 360.54, Closing Price: 151.84, Percentage Change: -57.89%
Symbol: STK003, Opening Price: 461.25, Closing Price: 268.94, Percentage Change: -41.69%
Symbol: STK009, Opening Price: 373.08, Closing Price: 248.91, Percentage Change: -33.28%
Symbol: STK015, Opening Price: 450.42, Closing Price: 379.83, Percentage Change: -15.67%
Symbol: STK005, Opening Price: 164.94, Closing Price: 143.15, Percentage Change: -13.21%
Symbol: STK001, Opening Price: 279.15, Closing Price: 242.69, Percentage Change: -13.06%
Symbol: STK014, Opening Price: 418.1, Closing Price: 394.03, Percentage Change: -5.76%
Symbol: STK012, Opening Price: 286.09, Closing Price: 270.12, Percentage Change: -5.58%
Symbol: STK007, Opening Price: 431.25, Closing Price: 414.47, Percentage Change: -3.89%
Symbol: STK011, Opening Price: 283.71, Closing Price: 276.66, Percentage Change: -2.48%
Symbol: STK020, Opening Price: 345.03, Closing Price: 342.15, Percentage Change: -0.83%
Symbol: STK002, Opening Price: 381.3, Closing Price: 389.84, Percentage Change: 2.24%
Symbol: STK004, Opening Price: 433.5, Closing Price: 449.48, Percentage Change: 3.69%
Symbol: STK010, Opening Price: 309.94, Closing Price: 335.28, Percentage Change: 8.18%
Symbol: STK008, Opening Price: 315.46, Closing Price: 488.24, Percentage Change: 54.77%
Symbol: STK016, Opening Price: 182.32, Closing Price: 295.36, Percentage Change: 62.00%
Symbol: STK013, Opening Price: 112.08, Closing Price: 199.87, Percentage Change: 78.33%
Symbol: STK018, Opening Price: 236.29, Closing Price: 470.9, Percentage Change: 99.29%
Symbol: STK006, Opening Price: 226.63, Closing Price: 477.6, Percentage Change: 110.74%

Stock found: Symbol=STK005, Opening Price=164.94, Closing Price=143.15, Percentage Change=-13.21%
PS C:\Users\VIGNANAND RAO\OneDrive\Desktop\AI_coding>
```

```
Performance Comparison:
| Operation                    | Recommended Algorithm              | Justification                                                                          |
|------------------------------|------------------------------------|----------------------------------------------------------------------------------------|
| Sort by Percentage Change    | Heap Sort                          | Heap Sort has a time complexity of O(n log n) and is efficient for sorting large       |
|                              |                                    |datasets. It is an in-place sorting algorithm, which means it does not require          |
|                              |                                    |additional space, making it suitable for sorting stock data.                            |
|------------------------------|------------------------------------|----------------------------------------------------------------------------------------|
| Search by Stock Symbol       | Hash Map (Dictionary)              | Hash maps provide O(1) average time complexity for lookups, making it ideal for       |
|                              |                                    | searching specific stock symbols instantly. This is especially beneficial when the    |
|                              |                                    | dataset size increases, as it allows for efficient retrieval of stock data.            |
|------------------------------|------------------------------------|----------------------------------------------------------------------------------------|

Trade-offs:
- Heap Sort is more efficient than Python's built-in sorted() for large datasets, but it may not be as optimized for small datasets due to its overhead.
- Hash maps (dictionaries) provide faster lookups compared to linear search, but they require additional memory to store the hash map,
 which may not be ideal for very large datasets with limited memory resources.
```

JUSTIFICATION:

In the above task, we have successfully implemented Heap Sort for sorting of stocks and Hash Maps for searching of the required stock by using stock symbols.

Heap sort has a time complexity of O(n log n) and it is efficient for sorting large datasets.

Hask Map has a time complexity of O(1).